

Laboratorio 3 – String y StringBuilder

1 Introducción

En este laboratorio comenzaréis a trabajar con objetos de dos clases predefinidas de Java: **String** y **StringBuilder**.

Ambas clases permiten representar cualquier texto de longitud arbitraria escrito en cualquier idioma.

La clase String permite también presentar información textual por pantalla de manera eficaz, y procesar entradas desde el teclado. Son clases muy útiles que suelen usarse constantemente en muchos programas.

2 Objetivos

Los objetivos de esta sesión son los siguientes:

- Aprender algunas de las prestaciones más relevantes de las clases String y StringBuilder para manipular cadenas de caracteres.
- Aprender a gestionar la entrada de datos a través del teclado.
- Implementar algoritmos de búsqueda.

En el lenguaje Java, todo dato que no sea de un tipo primitivo (int, double, char, boolean, etc.) es un objeto. Como en cualquier otro lenguaje de programación, en Java los objetos se gestionan mediante variables.

Como habéis aprendido de las clases de teoría, un objeto es una instancia de una clase. Una clase define las propiedades (atributos) y el comportamiento (método) de los objetos que son instancias de ella. Java proporciona una gran cantidad de clases que están a su libre disposición a la hora de programar.

Hoy practicaréis con dos de ellas: String y StringBuilder.

3 La clase String

Los objetos String:

- Contienen una cadena de caracteres y permiten su gestión.
- Son **inmutables**. Eso quiere decir que, una vez creado el objeto con una cadena de caracteres, **los contenidos de dicha cadena no pueden alterarse**.

3.1 Creación de objetos String

La creación de todo objeto String incluye explícitamente a la cadena de caracteres que debe contener, dada su inmutabilidad. Existen dos formas distintas con las que se obtienen los mismos resultados:

```
String texto = "Esto es un ejemplo de creación de String con un literal sin el operador new";
```

```
String texto2 = new String("Esto es un ejemplo de creación de String con un literal con el operador new: como puede verse, es más largo");
```

Como podéis ver, para crear un objeto String no es necesario el uso del operador new. Esta y la creación con listado de literales en los arrays son las únicas excepciones a la regla.

Algo de nomenclatura: a lo que hay a la derecha del signo igual o pasado como argumento al constructor de String se le denomina literal.

3.2 Operador + de concatenación

Cuando uno de los operandos del operador + es una variable o un literal String, la expresión `operando1 + operando2` es una expresión de concatenación de cadenas de caracteres.

Al evaluar estas expresiones, si uno de los dos operandos no es un String, la máquina virtual de Java genera una representación textual del operando (si es un objeto de alguna otra clase, invocando un método llamado `toString()` que todas las clases incorporan y que tiene como objetivo fundamental generar un String que de cuenta de los contenidos -valores de los atributos fundamentalmente- del objeto. Por otra parte, si es por ejemplo el número 12, genera el String "12"), y es esa representación textual la que se concatena con el otro operando (que debe ser un String, como se ha dicho antes).

La operación de concatenación **siempre crea un nuevo objeto String, ya que un objeto String es inmutable, es decir, no puede modificarse su contenido.**

```
String parte1 = "Esto es un ";
String parte2 = "ejemplo de concatenación. Se crea un tercer String";
String concatenado = parte1 + parte2;
/*El contenido de concatenado es : "Esto es un ejemplo de concatenación. Se
crea un tercer String" */
```

Concatenando un String con la representación textual de un número:

```
String parte1 = "Esto es un ejemplo de concatenación con el número ";
String concatenado = parte1 + 123;
/*El contenido de concatenado es : "Esto es un ejemplo de concatenación con
el número 123" */
```

Concatenando un String con un carácter:

```
String parte1 = "Esto es un ejemplo de concatenación con el carácter ";
String concatenado = parte1 + 'c';
/*El contenido de concatenado es : "Esto es un ejemplo de concatenación con
el carácter c" */
```

Concatenaciones más complejas

```
int a = 2, b = 5;
int resultado = a*b
String texto = "El resultado de " + a + " * " + b + " es " + resultado +
".";
//La variable "texto" contiene la cadena "El resultado de 2 * 5 es 10. "
```

3.3 Métodos de la clase String

Cuando trabajéis con los métodos de la clase String debéis tener en cuenta lo siguiente:

- El objeto String que contiene al método invocado no ve alterados sus contenidos. Recordad: **los objetos String son inmutables.**
- Todo método que retorna un String, **es un método que crea un nuevo objeto String.** El objeto retornado es distinto al objeto que contiene el método invocado.

En esta sesión de laboratorio trabajaréis con los siguientes métodos:

- `public int length()`: devuelve el número de caracteres de la cadena.
- `public char charAt(int i)`: devuelve el carácter que ocupa la posición de índice i en la cadena. A la primera posición se le asigna el índice 0.
- `public String toUpperCase()`, y `public String toLowerCase()`: cada una de ellas genera un nuevo objeto String. La primera contiene todos los caracteres de la cadena original pasados a mayúsculas. La segunda, todos los caracteres de la cadena original pasados a

minúsculas.

- **public String[] split(String regex):** genera un array de Strings resultante de dividir el String original alrededor de las coincidencias de la expresión regular pasada como argumento. Si dicho argumento es un String cuyo contenido es el espacio en blanco (" "), por ejemplo, los Strings del array retornado serán las palabras contenidas en el String original.

```
String frase = "Demostración de funcionamiento de split()";
```

```
String[] palabras = frase.split(" ");
```

Tras la ejecución de las dos instrucciones anteriores, el array `palabras` contendría referencias a cada una de las palabras del String `frase`.

`palabras[0]` sería "Demostración", `palabras[1]` sería "de", `palabras[2]` sería "funcionamiento", `palabras[3]` sería "de", y `palabras[4]` sería "split()".

- **public int compareTo(String anotherString):** compara dos Strings lexicográficamente. Devuelve un entero positivo si el String que contiene el método es lexicográficamente mayor que el String pasado con el argumento `anotherString`; devuelve 0 si los dos Strings contienen la misma cadena de caracteres; finalmente devuelve un entero negativo si el String que contiene el método es lexicográficamente menor que el String pasado con el argumento `anotherString`.
- **public static String valueOf(int i):** crea y retorna un String que contiene la representación textual del entero pasado como argumento (si el entero es 123, crea y retorna el String "123", por ejemplo).
- **public static String valueOf(double d):** crea y retorna un String que contiene la representación textual del real pasado como argumento (si el real es 123.4, crea y retorna el String "123.4", por ejemplo).
- **public String trim():** crea un nuevo String y copia en él los contenidos del String sobre el que se ha invocado el método excepto cualquier espacio blanco que hubiera antes del primer carácter no blanco y cualquier espacio blanco que hubiera después del último carácter no blanco.

La clase String incorpora también algunos métodos que permiten comparar contenidos de cadenas de caracteres (como **public boolean equals(Object anObject)**). Estos métodos se explican con detalle en la sección 5.

4 La clase StringBuilder

Esta clase permite gestionar cadenas de caracteres mutables (aquellas cuyos contenidos pueden cambiar). Los objetos `StringBuilder` gestionan internamente un buffer en el que podemos ir depositando y/o modificando caracteres.

En aquellas ocasiones en que hayamos de modificar frecuentemente las cadenas de caracteres es mejor utilizar esta clase que String.

En <http://www.edu4java.com/es/java/string-stringbuilder-stringBuffer-java.html> se muestra un pequeño programa que compara la eficiencia de `StringBuilder` y `String` en la concatenación de cadenas (recordad que cada vez que se concatenan dos objetos String se crea uno nuevo).

4.1 Creación de objetos StringBuilder

Los objetos `StringBuilder` se crean como cualquier otro objeto de Java.

```
StringBuilder texto = new StringBuilder("Esto es un ");
```

Observad que siempre debe utilizarse el operador `new`.

4.2 Métodos de la clase StringBuilder

A continuación se muestran los métodos más relevantes:

- **public StringBuilder append(CharSequence s):** concatena a la cadena de

caracteres contenida en el objeto `StringBuilder` que contiene al método, la cadena de caracteres contenida en el argumento `s`. El argumento `s` puede ser un objeto `StringBuilder` o un objeto `String`. El método devuelve el `StringBuilder` que contiene la cadena resultante de la concatenación, lo que permite realizar invocaciones en cascada como la que se muestra a continuación.

```
StringBuilder texto = new StringBuilder("Esto es un ")
    .append(("ejemplo de "))
    .append("concatenación por ").append("encadenamiento de ")
    .append("invocaciones al ").append("método append() ")
    .append("de StringBuilder");
```

- **`public StringBuilder append(int i), public StringBuilder append(double d)`**: concatenan a la cadena de caracteres contenida en el objeto `StringBuilder` que contiene al método, la representación textual del número entero `i` y del número real `d`, respectivamente.
- **`public String toString()`**. Este método crea un nuevo `String` cuya cadena de caracteres es una copia de la cadena de caracteres del objeto `StringBuilder`.
- Varios métodos `insert`, que permiten insertar cadenas de caracteres. Acceded a la documentación en línea de esta clase.
- Asimismo, `StringBuilder` comparte también un subconjunto de métodos con `String`. En general, **siempre es muy conveniente revisar la documentación de cualquier clase incluida en la distribución estándar de Java cuando vayáis a trabajar con ella**, y `String` y `StringBuilder` no son una excepción.

5 Comparando cadenas de caracteres

Puesto que los `Strings` son objetos, no es posible compararlos involucrando a las variables que los referencian en expresiones que incluyen operadores relacionales (`==`, `<`, `>`, etc.).

Las operaciones que impliquen comparaciones de `Strings` las realizan métodos de la clase `String`.

Todas las clases de la distribución estándar de Java incorporan el método siguiente:

- **`public boolean equals(Object o)`**: este método admite como argumento una referencia a un objeto de cualquier clase (cuando estudiéis herencia os presentaremos los detalles de la clase `Object` y entenderéis su papel en Java; de momento debéis creer que su aparición en una lista de argumentos de un método significa que ese argumento puede ser una referencia a un objeto de cualquier clase). En el caso de las clases `String` y `StringBuilder`, este método devuelve `true` si la cadena de caracteres que gestiona el objeto sobre el que se ha invocado el método y la cadena de caracteres que gestiona el argumento `o` tienen los mismos contenidos, y `false` en caso contrario.

Por su parte, la clase `String` también incorpora los métodos que siguen:

- **`public boolean equalsIgnoreCase(String anotherString)`**: hace lo mismo que el método anterior pero tratando como caracteres iguales a las mayúsculas y las minúsculas (es decir, considera iguales a los caracteres `A` y `a`, por ejemplo).
- **`public int compareTo(String anotherString)`**: este método devuelve:
 - un valor entero positivo si la cadena de caracteres que gestiona el objeto sobre el que se ha invocado el método es lexicográficamente mayor que la cadena de caracteres que gestiona el argumento `anotherString`;
 - un valor entero negativo si la cadena de caracteres que gestiona el objeto sobre el que se ha invocado el método es lexicográficamente menor que la cadena de caracteres que gestiona el argumento `anotherString`;
 - 0 si la cadena de caracteres que gestiona el objeto sobre el que se ha invocado el método tiene los mismos contenidos que la cadena de caracteres que gestiona el argumento `anotherString`.

Lexicográficamente mayor debe interpretarse como alfabéticamente posterior: “bondad” es lexicográficamente mayor que “animadversión”.

Existe una variante de `compareTo` que lleva a cabo el mismo trabajo sin hacer distinciones entre mayúsculas y minúsculas: `compareToIgnoreCase`.

6 La clase Scanner

Esta clase se encuentra en el package `java.util` y vamos a utilizarla para hacer que nuestros programas lean datos desde el teclado.

6.1 Importando clases

Cuando dentro del código de una clase A de Java se hace uso de otra clase B que no está ni en el mismo package que A ni en el package `java.lang`, hay que incluir una sentencia `import` al comienzo del archivo para informar al compilador y a la máquina virtual de Java del lugar en el que encontrarán el archivo que define la clase B. Imaginemos que la clase B está en el package `edu.upc.etsetb.poo.laboratorio.sesion3.apartadox`; la sentencia de importación de la clase B sería:

```
import edu.upc.etsetb.poo.laboratorio.sesion3.apartadox.B ;
```

6.2 Leyendo líneas desde el teclado

En esta sesión aprenderéis a hacer que el programa lea líneas desde el teclado y que las deposite en una objeto `String`.

```
Scanner lector = new Scanner(System.in) ;  
// Crea la variable lector para leer líneas  
String linea = lector.nextLine();  
//Lee los caracteres que entremos por teclado hasta que  
// pulsemos <Enter>.  
// Cuando lo hagamos, dejará de leer del teclado y depositará todos  
// los que hayamos entrado en el String línea.
```

Cada vez que se invoca el método `nextLine()`, el programa queda a la espera de que el usuario escriba una línea de texto y pulse la tecla de retorno. Cuando eso sucede, devuelve la línea de texto escrita (sin el carácter de final de línea).

Por las características de los objetos `Scanner`, normalmente un programa necesita solamente uno de ellos. Invocando sobre él el método `nextLine()` tantas veces como necesite, pueden leerse tantas líneas como sea preciso.

7 Ejercicios

Cread un nuevo proyecto para esta sesión.

7.1 Invirtiendo cadenas

Cread el package `edu.upc.etsetb.poo.laboratorio.sesion3.apartado1`. Cread en él la clase `InvertirCadena`. Cread en esa clase el método `main` que:

1. Cree un `String` compuesto de varias palabras ("Escola Tècnica Superior Enginyeria Telecomunicació Barcelona", por ejemplo).
2. Genere un nuevo `String` que contenga el `String` anterior invertido y con todas las letras en minúsculas.

Programad el paso 2 en dos versiones (en el mismo `main`, una detrás de otra): usando `String` y el operador `+` de concatenación, y usando `StringBuilder` y su método `append()`.

7.2 Comparando prestaciones de String y StringBuilder

Cread el package **edu.upc.etsetb.poo.laboratorio.sesion3.apartado2**. Cread en él la clase **StringVsStringBuilder**.

Acudid a la página web <http://www.edu4java.com/es/java/string-stringbuilder-stringBuffer-java.html> y copiad los métodos `main()`, `genWithStringBuilder()` y `genWithString()`. Compilad y ejecutad el programa. Observad cuánto tarda en acabar el método `genWithString()` y cuánto tarda en acabar el método `genWithStringBuilder()`. Ambos hacen lo mismo, pero el primero usa Strings y el operador + de concatenación, mientras que el segundo usa **StringBuilder** y su método `append()`.

7.3 Entrando líneas desde el teclado.

Cread el package **edu.upc.etsetb.poo.laboratorio.sesion3.apartado3**. Cread en él la clase **SaludarNombre**.

Escribid en esa clase el siguiente código y ejecutadlo. Observaréis que el programa presenta en la consola la pregunta "Com et dius?" y que al entrar por teclado vuestro nombre presentará en la consola el saludo. Para comprobar qué está pasando, podéis poner un breakpoint en la línea que presenta el saludo final y lanzar una sesión de depuración. Cuando el programa se detenga en dicho breakpoint, podréis ver cómo la variable `text` es un **String** que contiene la cadena que habéis introducido por el teclado.

```
import java.util.Scanner;

public class SaludarNom {
    public static void main (String[] args) {
        Scanner lector = new Scanner(System.in);
        System.out.print("Com et dius? ");
        String text = lector.nextLine();
        System.out.println("Hola " + text + "!");
    }
}
```

7.4 Generando acrónimos.

Cread el package **edu.upc.etsetb.poo.laboratorio.sesion3.apartado4**. Cread en él la clase **GeneradorAcronimo**.

Reutilizad el código de la clase del apartado anterior para que ahora una vez leída la línea del teclado, el programa muestre por pantalla el acrónimo de lo que hayáis tecleado. Así, si tecleáis ("Escola Tècnica Superior Enginyeria Telecomunicació Barcelona" la reacción del programa debe ser mostrar por consola "ETSETB".

Os sugerimos que probéis primero a asignar a crear una variable **String** con la cadena "Escola Tècnica Superior Enginyeria Telecomunicació Barcelona"; escribid el código que genere el acrónimo, y sólo cuando éste funcione, sustituid la línea de creación de la variable **String** con el código que lea la línea desde el teclado. Así no tendréis que teclear una y otra vez lo mismo cuando el código no funcione e intentéis arreglarlo.

7.5 Búsqueda lineal sobre Strings creados con literales.

Cread el package **edu.upc.etsetb.poo.laboratorio.sesion3.apartado5**. Cread en él la clase **BuscadorDePalabras**.

Cread dentro un método `main()` que realice las siguientes tareas:

1. Crear un **String** que contenga varias palabras separadas por un espacio en blanco.
2. Crea un segundo **String** que contenga una palabra de las que había en el **String** anterior.
3. Busque la palabra en el **String** inicial e informe por consola de si está o no en el **String**

Os sugerimos que utilices los métodos `split()` y `equals()` antes comentados.

Para comprobar que vuestro código funciona correctamente, tenéis que lanzar dos ejecuciones: una buscando una palabra que esté en el String inicial, y otra buscando una palabra que no lo esté. Ello os obligará a cambiar la instrucción del paso 2.

7.6 Búsqueda lineal sobre Strings introducidos por el usuario desde el teclado.

Cuando el programa anterior os funcione correctamente sustituid los pasos 1 y 2 por código que permita al programa pedirle al usuario que introduzca el String con las palabras y la palabra buscada. Usad la clase Scanner.

7.7 Búsqueda dicotómica sobre arrays de Strings ordenados creados con literales.

La búsqueda dicotómica es un algoritmo de búsqueda adecuado cuando los contenidos del array están ordenados.

A continuación se detalla el algoritmo. Dado un cierto dato buscado,

Declarar tres variables enteras, `primera` y `ultima`, y `centro`.

Inicializar variable `primera` a 0 y `ultima` al índice de la última posición del array

Mientras `primera <= ultima`

Inicializar `centro` al resultado de realizar la división entera $(ultima+primera)/2$.

Hacer `ref=array[centro]`

Si `ref` es lo buscado finalizar porque se ha encontrado lo buscado

Si `ref` es menor que lo buscado

`primera = centro + 1`

Sino

`ultima = centro - 1`

FinMientras

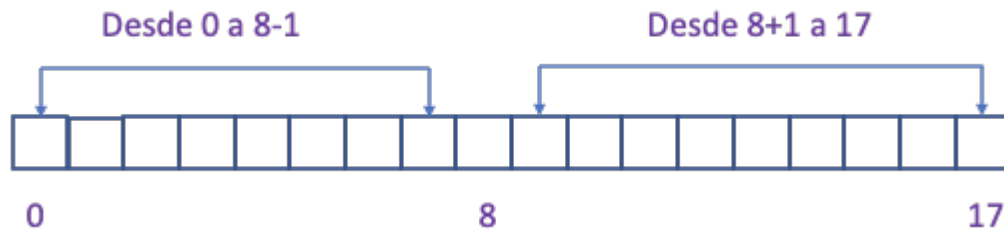
Notificar que no se ha encontrado lo buscado

Este algoritmo divide, en cada iteración, la zona del array en la que busca el dato, en dos mitades.

Observad, en la variable `ref` tenéis un valor que está en la posición cuyo índice es `centro`; si lo buscado es mayor que lo que hay en (o referencia) `ref`, como el array está ordenado, lo buscado solo puede estar a la derecha de la posición con índice `centro`. En consecuencia, la siguiente iteración habrá que limitarla a la zona del array comprendida entre las posiciones con índices `centro+1` y `ultima`.

Por el contrario, si lo buscado es menor que lo que hay en (o lo que referencia) `ref`, lo buscado solo puede estar a la izquierda de la posición con índice `centro`. En consecuencia, la siguiente iteración habrá que limitarla a la zona del array comprendida entre las posiciones con índices `primera` y `centro-1`.

La figura que sigue muestra las dos zonas en las que el array queda dividido a partir de su posición central. Observad cuáles son los límites de cada una de las dos zonas:



El bucle por tanto, va disminuyendo los valores de `ultima` y aumentando los valores de `primera`. Si el dato buscado no está en el array, llega un momento en que, o bien `ultima` es igual a `primera`, o bien `primera > ultima`. El cumplimiento de cualquiera de esas dos condiciones determina el final infructuoso de la búsqueda y se concluye que el dato no está en el array.

Cread el package `edu.upc.etsetb.poo.laboratorio.sesion3.apartado6`. Cread en él la clase `BuscadorDicotomicoDePalabras`.

Cread dentro un método `main()` que realice las siguientes tareas:

1. Crear un String que contenga varias palabras separadas por un espacio en blanco **y que estén ordenadas alfabéticamente en el String**.
2. Crea un segundo String que contenga una palabra de las que había en el String anterior.
3. Busque la palabra utilizando la búsqueda dicotómica e informe por consola de si está o no en el String

Observad que en la descripción del algoritmo hemos escrito la siguiente condición: "Si `ref` es menor que lo buscado", en lugar de "Si `ref < lo buscado`".

Ello es así porque la segunda condición es válida solamente para datos para los que el operador `<` está definido, números en esencia. Sin embargo la búsqueda la realizáis sobre un array de Strings, lo que buscáis no es un número, sino un String, y el operador `>` no opera sobre cadenas de caracteres. Sin embargo uno de los métodos String que os hemos presentado antes, permite saber si la cadena de caracteres de un cierto String es igual, lexicográficamente mayor o lexicográficamente menor que la cadena de caracteres de otro String: ese es el método que tenéis que usar.